

v6 — openpilot on the car

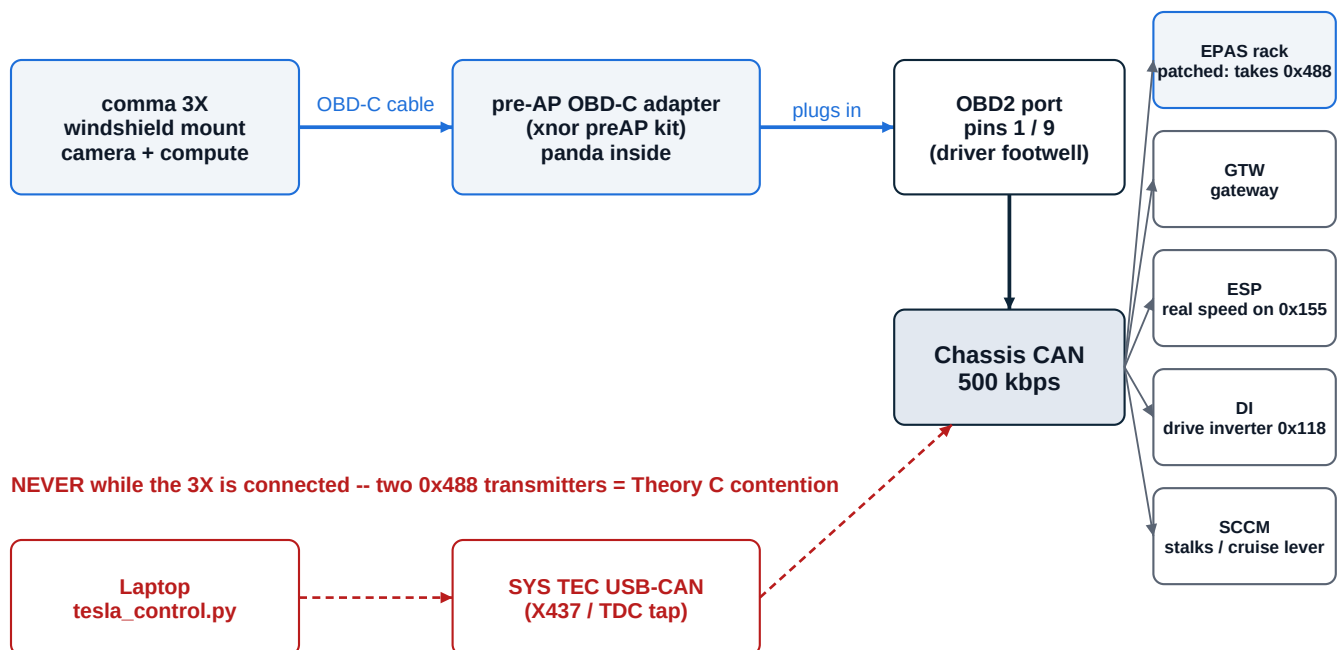
Connection plan, test plan, and commercial cost analysis for running a comma 3X with a pre-AP openpilot fork on the 2013 Tesla Model S.

v6 is a different family from v4/v5: the rack-control programs make the laptop the brain; v6 makes the comma 3X the brain and retires our code to a bench tool. The two are mutually exclusive on the bus — the device and `tesla_control.py` both transmit 0x488 DAS_steeringControl, and two transmitters on one arbitration ID is exactly the Theory C contention confirmed in May 2026 (PROJECT_MEMORY.md §8).

Companion documents: v6/V6_PLAN.md (research, with citations), v6/INSTALL_GUIDE.md (step-by-step), v6/ANALYSIS_PROMPT1.md (option comparison answering Charlie's prompt1.md).

1. System architecture

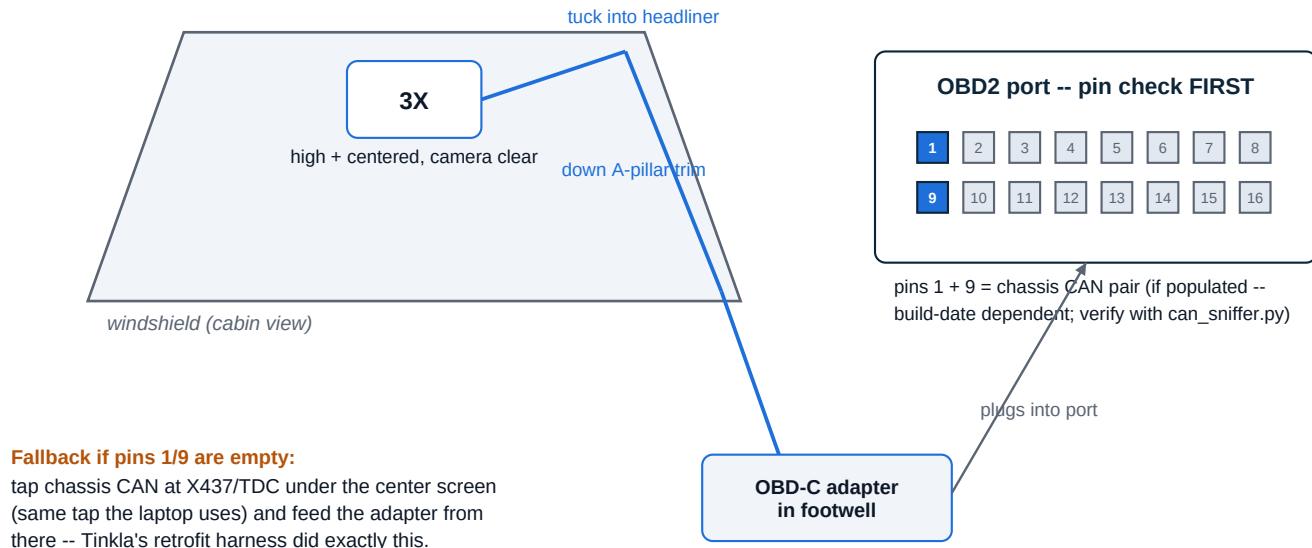
The comma 3X mounts on the windshield and reaches chassis CAN through the pre-AP OBD-C adapter in the OBD2 port. The EPAS rack is already flashed with the gregjhogan patch (done by Jordan via the BogGyver UI), so it accepts 0x488 steering commands directly — the same mechanism the v4 laptop control uses today.



One transmitter at a time: before plugging in the 3X harness, physically disconnect the SYS TEC adapter. The v4.2 bus diagnostic panel's 0x488 RX-rate readout is the detection tool if there is ever any doubt.

2. Physical connection

First physical task: open the OBD2 port cover in the driver footwell and check whether pins 1 and 9 are populated — on this platform that pair carries chassis CAN, but population depends on build date (our car is a post-May-31 2013 build, which is why this is a check and not an assumption). Photograph the port for the log either way.



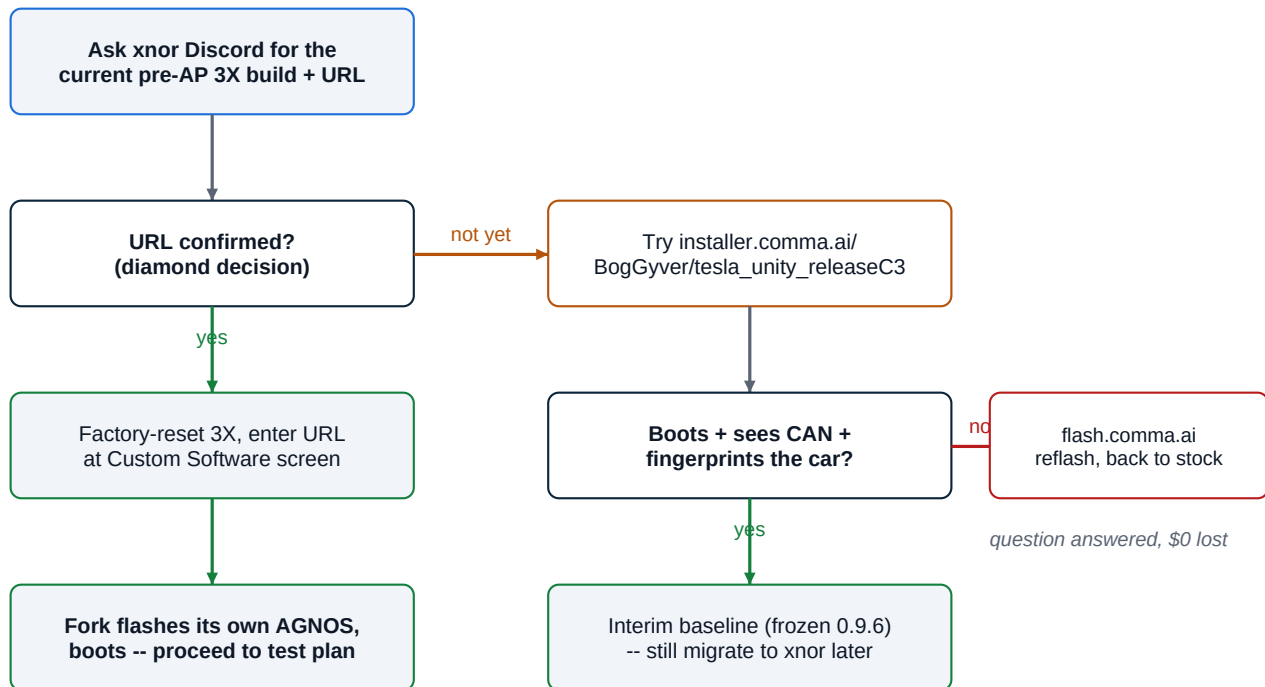
Verification before first power-on: with the adapter wired but the 3X NOT yet connected, put the laptop sniffer (can_sniffer.py) on the same pair and confirm real chassis traffic (0x370 EPAS, 0x155 ESP, 0x118 DI at their known rates). That proves the tap is chassis CAN and not body/diag CAN before any new device transmits on it.

Parts list

Item	Source	Price	Status
comma 3X	comma.ai	\$1,250 list (seen ~\$1,050 on sale)	in hand
pre-AP OBD-C harness kit	xnor.shop (ships from Germany)	unpublished — confirm at order time (est. \$100–250)	to order after pin check
OBD-C extension cable ~2 m	included in xnor kit	—	—
EPAS firmware patch	gregjhogan/tesla-pre-ap-epas-patch	\$0	ALREADY FLASHED — do not re-flash
comma pedal interceptor (optional, later)	comma / DIY	~\$300 (est.)	extends ACC below ~18 mph; shared goal with v5 plan

3. Software install

Decision flow from ANALYSIS_PROMPT1.md. The recommended target is the actively maintained xnor/Loetkolben build (the commaai community wiki's documented path for pre-AP Model S on a comma 3X); the frozen BogGyver branch is a zero-cost spike worth one afternoon, never two days. No manual AGNOS step exists anywhere in this flow — every fork pins its AGNOS version in launch_env.sh and flashes it automatically on first boot (BogGyver C3 pins 9.1; xnor-c3 pins 12.8). The device currently runs stock AGNOS 18.4, which is fine: the installer handles the transition. The BogGyver installer URL is built from github.com/BogGyver/openpilot, branch `tesla_unity_releaseC3` — that branch is the source of the AGNOS pin and the place to read the code before installing.



Never: `installer.comma.ai/commaai/agnos8` (branch does not exist) · never re-flash the EPAS patch (already done)

4. Test plan

Phases are gated: do not start a phase until the previous phase's acceptance criteria are met and logged. Save artifacts to `field_testing/sessions/` with the same note discipline as the v4 sessions.

Phase	Where	What	Acceptance criteria
T0 pin check	car, parked	Inspect OBD2 pins 1/9; sniff the pair with <code>can_sniffer.py</code>	Photo of port; capture showing 0x370/0x155/0x118 at known rates (or decision to use X437 fallback)
T1 bench boot	bench, no car	Factory reset; install fork via custom-software URL	Fork boots reliably; Tesla preAP settings screen present; EPAS screen reports patched state WITHOUT prompting a re-flash
T2 passive drive	car, human driving	Harness in, laptop disconnected; drive normally, never engage	Device fingerprints / accepts manual pre-AP selection; speed + steering angle on screen match reality; full route recorded; no DTCs, no EAC complaints, no flicker
T3 first engage	open road, > 18 mph	Engage via cruise stalk, hands on wheel; test override and brake disengage	Smooth lateral hold; instant disengage on steering override AND on brake; repeatable across 3+ engagements; clean log
T4 hardening	daily driving	Longer routes, tuning, decide on pedal interceptor	No safety events across multiple drives; go/no-go on low-speed ACC hardware

Standing rules: EAC flicker or cluster complaints during T2 = pull the harness and capture a log before anything else (bus-contention smell). The fork asking to flash EPAS = stop, investigate detection; the rack is already patched and working. `tesla_control.py` stays retired from the car for as long as the 3X is wired in.

Known limits to not chase as bugs: no ACC below ~18 mph without a pedal interceptor (no radar, no CAN throttle on pre-AP); no stop-and-go or hard braking authority (vacuum brakes, no iBooster); no standstill steering (rack speed gate applies to openpilot exactly as it does to our laptop).

5. Could we sell this? Cost analysis

Question asked: if the v6 integration works, is there a business in selling it — and how profitable would it be given the integration cost? Short answer: as a product business, no; as a small local install service, marginal at best. The numbers below show why. Everything marked (est.) is an estimate, not a verified figure.

5.1 The market

pre-AP Model S built (2012 - Oct 2014, worldwide)
~55,000 (est.)

still on the road, 12-14 years on (est. 65-75%)
~38,000

owners who would modify steering firmware on their car
~1-2% -> ~400-800

reachable + willing to pay a US installer (est.)
~40-120 lifetime

All figures estimates; fleet size from public Model S delivery numbers, conversion rates are judgment calls labelled as such.

The addressable market is small and shrinking: the newest eligible car was built in October 2014, every sale requires an owner willing to have their steering firmware modified, and the serious DIY slice of that population can already buy every piece themselves (comma sells the 3X, xnor sells the harness and maintains the software). Our sellable value-add is only the integration labor, the EPAS patch service, and a tested configuration.

5.2 Integration cost per car (turnkey install)

Cost item	Low	High	Notes
comma 3X (at our cost)	\$1,050	\$1,250	no dealer/volume pricing program exists (est.)
xnor pre-AP harness kit + shipping	\$130	\$300	price unpublished; DE shipping + duties (est.)
Install labor: mount, route, pin check, possible X437 retrofit tap	3 h	8 h	at \$100/h shop rate: \$300–\$800
EPAS patch flash (skip if already patched)	1 h	3 h	includes backup of original firmware; carries brick risk
Road validation (T2 + T3 equivalent)	2 h	4 h	\$200–\$400; non-negotiable before handover
Support reserve per car (first year)	\$150	\$400	fork updates break things; customers call us, not xnor (est.)
Total cost per delivered car	~\$1,930	~\$3,450	before insurance/liability (see 5.4)

5.3 Pricing scenarios and margin

Scenario	Price	Margin/car	Annual volume (est.)	Annual profit (est.)
A. Turnkey install (device + harness + install + validation)	\$2,950	\$400–\$1,000 (mid ~\$700)	5–15 cars	\$3,500–\$10,500
B. Install-only service (customer brings 3X + kit)	\$650	\$250–\$400	5–20 cars	\$1,500–\$8,000
C. Documentation / kit-less (publish guides, donations)	\$0	\$0	—	\$0 — but also zero liability

Break-even reality check on Scenario A: at a mid-case ~\$700 margin, ten installs a year clears about \$7,000 — before insurance, before a single warranty callback, before any marketing. One liability event, one bricked EPAS requiring a used rack (~\$400–\$900 + labor), or one customer whose fork update breaks at 2 a.m. erases weeks of margin. Volume cannot rescue it: the funnel above caps lifetime demand in the tens to low hundreds of units, total.

5.4 The disqualifying risks

Risk	Severity	Why
Liability	disqualifying	Selling modified steering firmware + an aftermarket self-steering install on customer cars. comma ships as L2 driver assistance under its own terms; a small installer has no such shield. One incident outweighs all revenue.
Platform dependence	high	The product is really comma's device + xnor's software. Either party changing course (3X EOL, fork abandoned -- which already happened once with BogGyver) strands customers we are on the hook for.
Shrinking market	high	Fleet ages out; every year the funnel narrows. No new supply of pre-AP cars, ever.
EPAS patch brick risk	medium	Flashing customer racks at scale means eventually bricking one. Backup/restore mitigates but does not eliminate.

5.5 Verdict

Not a business. The integration cost per car (~\$1,900–\$3,400) against a defensible price ceiling around \$3,000 leaves a margin that is thin in the best case and negative after one bad event, on a market measured in dozens of lifetime customers — all of it carrying steering-system liability we cannot insure away at this scale. If any commercial path is worth keeping open, it is Scenario C: publish the v6 documentation, build reputation in the xnor/openpilot community, and treat paid one-off installs for nearby owners as occasional shop work (Scenario B pricing) rather than a product line. The honest framing: v6 is worth doing for our car and for the project's openpilot end-goal — not for revenue.